==================================

CAPSTONE PROJECT

NOTEBOOK 46: FINAL FULL-PROJECT CONSOLIDATION

==================================

Purpose:

This notebook freezes the final project state and consolidates

the main modelling, pipeline, demo, and delivery outputs into

report-ready tables.

The goal is to:

1. Freeze the final candidate-150 benchmark configuration

2. Freeze the final full-161 Stage-1 + Stage-2 configuration

3. Consolidate the main project results into clean summary tables

4. Build deliverable checklists aligned to the proposal

5. Create final recommendation, limitations, and next-step tables

6. Save report-ready consolidation outputs

==================================

In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import os
import json
import numpy as np
import pandas as pd

SEED = 42
np.random.seed(SEED)

print("Seed set to:", SEED)
```

```
Seed set to: 42
```

In [2]:

```
# ============================================================
# 2. PATHS AND HELPERS
# ============================================================

PROCESSED_DIR = "../data/processed"
os.makedirs(PROCESSED_DIR, exist_ok=True)

print("Processed directory:", os.path.abspath(PROCESSED_DIR))

def safe_read_csv(path):
    if os.path.exists(path):
        df = pd.read_csv(path)
        print(f"Loaded: {path} -> {df.shape}")
        return df
    else:
        print(f"Missing: {path}")
```

```
        return None

def safe_read_json(path):
    if os.path.exists(path):
        with open(path, "r") as f:
            obj = json.load(f)
        print(f"Loaded: {path}")
        return obj
    else:
        print(f"Missing: {path}")
        return None
```

```
Processed directory: e:\SCHOOL\Masters\Capstone_FMA_Project\data\processed
```

In [3]:

```
# ============================================================
# 3. LOAD CONSOLIDATION INPUTS FROM PRIOR NOTEBOOKS
# ============================================================

# Full-161 pipeline
full161_pipeline_summary_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_end_to_end_pipeline_summary.csv"
)

# Single-file deployment demo
single_file_summary_json = safe_read_json(
    f"{PROCESSED_DIR}/full161_single_file_inference_summary.json"
)

# Random batch demo
batch_demo_summary_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_batch_inference_summary.csv"
)

# Targeted rare-tail batch demo
targeted_rare_tail_summary_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_targeted_rare_tail_batch_summary.csv"
)

# Rare-tail reranker comparison
stage2_reranker_test_comparison_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_stage2_reranker_test_comparison.csv"
)

stage2_reranker_final_summary_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_stage2_reranker_final_summary.csv"
)

stage2_reranker_label_level_df = safe_read_csv(
    f"{PROCESSED_DIR}/full161_stage2_reranker_label_level_summary.csv"
)

stage2_reranker_targeted_test_df = safe_read_csv(
```

```
    f"{PROCESSED_DIR}/full161_stage2_reranker_targeted_test_results.csv"
)
```

```
Loaded: ../data/processed/full161_end_to_end_pipeline_summary.csv -> (6, 3)
Loaded: ../data/processed/full161_single_file_inference_summary.json
Loaded: ../data/processed/full161_batch_inference_summary.csv -> (9, 2)
Loaded: ../data/processed/full161_targeted_rare_tail_batch_summary.csv -> (11, 2)
Loaded: ../data/processed/full161_stage2_reranker_test_comparison.csv -> (2, 9)
Loaded: ../data/processed/full161_stage2_reranker_final_summary.csv -> (6, 3)
Loaded: ../data/processed/full161_stage2_reranker_label_level_summary.csv -> (10, 6)
Loaded: ../data/processed/full161_stage2_reranker_targeted_test_results.csv -> (35,
13)
```

In [4]:

```
# ============================================================
# 4. FREEZE FINAL PROJECT CONFIGURATION
# ============================================================
# Frozen from the final modelling path completed in prior notebooks.

final_project_config = {
    "dataset": "FMA-Large",
    "working_label_space_stage1": 150,
    "full_label_space": 161,
    "stage1_primary_system_name": "Expanded Hybrid Global Threshold",
    "stage1_structured_weight": 0.1,
    "stage1_audio_weight": 0.9,
    "stage1_threshold": 0.2,
    "stage2_router_name": "Rare-tail fallback router",
    "stage2_final_strategy": "baseline_prior",
    "stage2_anchor_trigger_threshold": 0.05,
    "stage2_top_k": 1,
    "inventory_only_rare_tail_labels": 3,
    "hierarchy_fallback_rare_tail_labels": 10
}

print("Final project configuration:")
print(json.dumps(final_project_config, indent=2))
```

```
Final project configuration:
{
  "dataset": "FMA-Large",
  "working_label_space_stage1": 150,
  "full_label_space": 161,
  "stage1_primary_system_name": "Expanded Hybrid Global Threshold",
  "stage1_structured_weight": 0.1,
  "stage1_audio_weight": 0.9,
  "stage1_threshold": 0.2,
  "stage2_router_name": "Rare-tail fallback router",
  "stage2_final_strategy": "baseline_prior",
  "stage2_anchor_trigger_threshold": 0.05,
  "stage2_top_k": 1,
  "inventory_only_rare_tail_labels": 3,
  "hierarchy_fallback_rare_tail_labels": 10
}
```

In [5]:

```python
# ============================================================
# 5. FREEZE FINAL CANDIDATE-150 BENCHMARK TABLE
# ============================================================
# These are the final candidate-150 results frozen from the modelling sequence.

candidate150_final_df = pd.DataFrame([
    {
        "System Level": "Candidate-150",
        "Model Family": "Structured",
        "Model Name": "Final Structured Baseline",
        "Config": "OVR SGD Hinge + per-label decoding",
        "Micro F1": 0.163700,
        "Macro F1": 0.082346,
        "Samples F1": 0.164799,
        "Use Case": "Reference branch"
    },
    {
        "System Level": "Candidate-150",
        "Model Family": "Audio",
        "Model Name": "Expanded Audio CNN - Tuned Threshold",
        "Config": "Threshold = 0.20",
        "Micro F1": 0.381704,
        "Macro F1": 0.040540,
        "Samples F1": 0.379117,
        "Use Case": "Strong single-branch benchmark"
    },
    {
        "System Level": "Candidate-150",
        "Model Family": "Hybrid",
        "Model Name": "Expanded Hybrid Global Threshold",
        "Config": "Structured 0.1 | Audio 0.9 | Threshold 0.20",
        "Micro F1": 0.391023,
        "Macro F1": 0.056410,
        "Samples F1": 0.386741,
        "Use Case": "Final benchmark model"
    },
    {
        "System Level": "Candidate-150",
```

```
        "Model Family": "Hybrid",
        "Model Name": "Expanded Hybrid Per-Label + Best Cap",
        "Config": "Conservative decoding | Cap = 3",
        "Micro F1": 0.375856,
        "Macro F1": 0.045106,
        "Samples F1": 0.376832,
        "Use Case": "Conservative optional mode"
    }
])

print("Frozen candidate-150 final benchmark table:")
display(candidate150_final_df)
```

```
Frozen candidate-150 final benchmark table:
```

	System Level	Model Family	Model Name	Config	Micro F1	Macro F1	Samples F1	Use Case
0	Candidate-150	Structured	Final Structured Baseline	OVR SGD Hinge + per-label decoding	0.163700	0.082346	0.164799	Reference branch
1	Candidate-150	Audio	Expanded Audio CNN - Tuned Threshold	Threshold = 0.20	0.381704	0.040540	0.379117	Strong single-branch benchmark
2	Candidate-150	Hybrid	Expanded Hybrid Global Threshold	Structured 0.1 | Audio 0.9 | Threshold 0.20	0.391023	0.056410	0.386741	Final benchmark model
3	Candidate-150	Hybrid	Expanded Hybrid Per-Label + Best Cap	Conservative decoding | Cap = 3	0.375856	0.045106	0.376832	Conservative optional mode

In [6]:

```
# ============================================================
# 6. FREEZE FINAL FULL-161 SYSTEM TABLE
# ============================================================

full161_final_df = pd.DataFrame([
    {
        "System Level": "Full-161",
        "Component": "Stage 1",
        "Name": "Expanded Hybrid Global Threshold",
        "Role": "Direct prediction of candidate labels",
        "Config": "Structured 0.1 | Audio 0.9 | Threshold 0.20",
        "Notes": "Primary benchmark prediction engine"
    },
```

```
    {
        "System Level": "Full-161",
        "Component": "Stage 2",
        "Name": "Rare-tail fallback router",
        "Role": "Fallback suggestion layer for sparse rare-tail genres",
        "Config": "baseline_prior | Anchor trigger threshold = 0.05 | Top-K = 1",
        "Notes": "Not a standalone strong classifier; used as a fallback/hint
layer"
    },
    {
        "System Level": "Full-161",
        "Component": "Inventory-only",
        "Name": "Rare-tail inventory-only labels",
        "Role": "Retained in taxonomy but not directly predicted",
        "Config": "3 labels",
        "Notes": "Insufficient support for reliable direct modelling"
    }
])

print("Frozen full-161 system table:")
display(full161_final_df)
```

```
Frozen full-161 system table:
```

	System Level	Component	Name	Role	Config	Notes
0	Full-161	Stage 1	Expanded Hybrid Global Threshold	Direct prediction of candidate labels	Structured 0.1 | Audio 0.9 | Threshold 0.20	Primary benchmark prediction engine
1	Full-161	Stage 2	Rare-tail fallback router	Fallback suggestion layer for sparse rare-tail...	baseline_prior | Anchor trigger threshold = 0....	Not a standalone strong classifier; used as a ...
2	Full-161	Inventory-only	Rare-tail inventory-only labels	Retained in taxonomy but not directly predicted	3 labels	Insufficient support for reliable direct model...

In [7]:

```
# ============================================================
# 7. BUILD FINAL PERFORMANCE SNAPSHOT TABLE
# ============================================================

# Pull what we can from loaded summary tables
random_batch_candidate_hit_rate = np.nan
random_batch_avg_candidate_labels = np.nan
random_batch_avg_rare_tail_suggestions = np.nan

if batch_demo_summary_df is not None:
    summary_map = dict(zip(batch_demo_summary_df["Metric"],
batch_demo_summary_df["Value"]))
```

```
    random_batch_candidate_hit_rate = summary_map.get("Candidate hit-any rate",
np.nan)
    random_batch_avg_candidate_labels = summary_map.get("Average predicted
candidate labels", np.nan)
    random_batch_avg_rare_tail_suggestions = summary_map.get("Average rare-tail
suggestions", np.nan)

targeted_candidate_hit_rate = np.nan
targeted_stage2_coverage = np.nan
targeted_rare_tail_hit_rate = np.nan
targeted_rare_tail_top1_rate = np.nan

if targeted_rare_tail_summary_df is not None:
    summary_map = dict(zip(targeted_rare_tail_summary_df["Metric"],
targeted_rare_tail_summary_df["Value"]))
    targeted_candidate_hit_rate = summary_map.get("Candidate hit-any rate", np.nan)
    targeted_stage2_coverage = summary_map.get("Stage-2 suggestion coverage",
np.nan)
    targeted_rare_tail_hit_rate = summary_map.get("Rare-tail hit-any rate", np.nan)
    targeted_rare_tail_top1_rate = summary_map.get("Rare-tail exact top-1 hit
rate", np.nan)

reranker_baseline_rate = np.nan
reranker_best_rate = np.nan

if stage2_reranker_test_comparison_df is not None:
    if stage2_reranker_test_comparison_df.shape[0] >= 1:
        reranker_baseline_rate = stage2_reranker_test_comparison_df.iloc[0]["Rare-
Tail Hit-Any Rate"]
    if stage2_reranker_test_comparison_df.shape[0] >= 2:
        reranker_best_rate = stage2_reranker_test_comparison_df.iloc[1]["Rare-Tail
Hit-Any Rate"]

performance_snapshot_df = pd.DataFrame([
    {
        "Area": "Candidate-150 benchmark",
        "Metric": "Best benchmark Micro F1",
        "Value": 0.391023,
        "Interpretation": "Final benchmark model for the candidate-150 modelling
phase"
    },
    {
        "Area": "Candidate-150 benchmark",
        "Metric": "Best conservative Micro F1",
        "Value": 0.375856,
        "Interpretation": "Cleaner optional conservative mode"
    },
    {
        "Area": "Full-161 random batch demo",
        "Metric": "Candidate hit-any rate",
        "Value": random_batch_candidate_hit_rate,
        "Interpretation": "Qualitative batch deployment performance on random
sample"
    },
    {
        "Area": "Full-161 random batch demo",
        "Metric": "Average predicted candidate labels",
```

```
        "Value": random_batch_avg_candidate_labels,
        "Interpretation": "Average number of Stage-1 direct predictions per sampled
track"
    },
    {
        "Area": "Full-161 random batch demo",
        "Metric": "Average rare-tail suggestions",
        "Value": random_batch_avg_rare_tail_suggestions,
        "Interpretation": "Average Stage-2 suggestions per sampled track"
    },
    {
        "Area": "Full-161 targeted rare-tail demo",
        "Metric": "Candidate hit-any rate",
        "Value": targeted_candidate_hit_rate,
        "Interpretation": "Stage-1 behaviour on rare-tail-containing tracks"
    },
    {
        "Area": "Full-161 targeted rare-tail demo",
        "Metric": "Stage-2 suggestion coverage",
        "Value": targeted_stage2_coverage,
        "Interpretation": "How often Stage-2 surfaced a suggestion on targeted
rare-tail tracks"
    },
    {
        "Area": "Full-161 targeted rare-tail demo",
        "Metric": "Rare-tail hit-any rate",
        "Value": targeted_rare_tail_hit_rate,
        "Interpretation": "Pre-reranker Stage-2 success rate on targeted rare-tail
tracks"
    },
    {
        "Area": "Full-161 targeted rare-tail demo",
        "Metric": "Rare-tail exact top-1 hit rate",
        "Value": targeted_rare_tail_top1_rate,
        "Interpretation": "Pre-reranker top suggestion accuracy on targeted rare-
tail tracks"
    },
    {
        "Area": "Stage-2 reranker comparison",
        "Metric": "Baseline rare-tail hit-any rate",
        "Value": reranker_baseline_rate,
        "Interpretation": "Baseline Stage-2 rare-tail test performance"
    },
    {
        "Area": "Stage-2 reranker comparison",
        "Metric": "Best Stage-2 rare-tail hit-any rate",
        "Value": reranker_best_rate,
        "Interpretation": "Final frozen Stage-2 rare-tail test performance"
    }
])

print("Final performance snapshot:")
display(performance_snapshot_df)
```

```
Final performance snapshot:
```

	Area	Metric	Value	Interpretation
0	Candidate-150 benchmark	Best benchmark Micro F1	0.391023	Final benchmark model for the candidate-150 mo...
1	Candidate-150 benchmark	Best conservative Micro F1	0.375856	Cleaner optional conservative mode
2	Full-161 random batch demo	Candidate hit-any rate	0.900000	Qualitative batch deployment performance on ra...
3	Full-161 random batch demo	Average predicted candidate labels	4.333333	Average number of Stage-1 direct predictions p...
4	Full-161 random batch demo	Average rare-tail suggestions	0.466667	Average Stage-2 suggestions per sampled track
5	Full-161 targeted rare-tail demo	Candidate hit-any rate	0.857143	Stage-1 behaviour on rare-tail-containing tracks
6	Full-161 targeted rare-tail demo	Stage-2 suggestion coverage	0.657143	How often Stage-2 surfaced a suggestion on tar...
7	Full-161 targeted rare-tail demo	Rare-tail hit-any rate	0.028571	Pre-reranker Stage-2 success rate on targeted ...
8	Full-161 targeted rare-tail demo	Rare-tail exact top-1 hit rate	0.028571	Pre-reranker top suggestion accuracy on target...
9	Stage-2 reranker comparison	Baseline rare-tail hit-any rate	0.285714	Baseline Stage-2 rare-tail test performance
10	Stage-2 reranker comparison	Best Stage-2 rare-tail hit-any rate	0.314286	Final frozen Stage-2 rare-tail test performance

In [8]:

```
# =============================================================
# 8. BUILD FINAL PROJECT DELIVERABLE CHECKLIST
# =============================================================

deliverables_df = pd.DataFrame([
    {
        "Deliverable Area": "Data ingestion and storage",
        "Deliverable": "MongoDB-backed metadata and label management layer",
        "Status": "Complete",
        "Notes": "Unified track and genre records created for the multi-label
phase"
    },
    {
        "Deliverable Area": "Data preparation",
        "Deliverable": "Cleaned and partitioned modelling-ready dataset",
        "Status": "Complete",
        "Notes": "Predefined training/validation/test splits preserved"
    },
    {
        "Deliverable Area": "Structured modelling",
        "Deliverable": "Structured multi-label baseline and improvements",
```

```
        "Status": "Complete",
        "Notes": "Multiple structured models evaluated and decoded"
    },
    {
        "Deliverable Area": "Audio modelling",
        "Deliverable": "Audio CNN multi-label benchmarks",
        "Status": "Complete",
        "Notes": "Pilot and expanded audio experiments completed"
    },
    {
        "Deliverable Area": "Hybrid modelling",
        "Deliverable": "Structured + audio hybrid benchmark",
        "Status": "Complete",
        "Notes": "Expanded hybrid benchmark frozen as main candidate-150 system"
    },
    {
        "Deliverable Area": "Full-genre extension",
        "Deliverable": "Full-161 Stage-1 + Stage-2 pipeline design",
        "Status": "Complete",
        "Notes": "Candidate labels predicted directly; rare-tail labels handled via
fallback"
    },
    {
        "Deliverable Area": "Deployment demo",
        "Deliverable": "Single-file inference demonstration",
        "Status": "Complete",
        "Notes": "One-file end-to-end full-161 inference completed"
    },
    {
        "Deliverable Area": "Deployment demo",
        "Deliverable": "Batch inference demonstration",
        "Status": "Complete",
        "Notes": "Random batch and targeted rare-tail batch demos completed"
    },
    {
        "Deliverable Area": "Comparative reporting",
        "Deliverable": "Comparative performance summary tables",
        "Status": "Complete",
        "Notes": "Project-level benchmark and fallback summaries consolidated"
    },
    {
        "Deliverable Area": "Interpretation",
        "Deliverable": "Final limitations and next-step framing",
        "Status": "Complete",
        "Notes": "Ready to translate into report and supervisor presentation"
    }
])

print("Deliverables checklist:")
display(deliverables_df)
```

```
Deliverables checklist:
```

	Deliverable Area	Deliverable	Status	Notes
0	Data ingestion and storage	MongoDB-backed metadata and label management l...	Complete	Unified track and genre records created for th...
1	Data preparation	Cleaned and partitioned modelling-ready dataset	Complete	Predefined training/validation/test splits pre...
2	Structured modelling	Structured multi-label baseline and improvements	Complete	Multiple structured models evaluated and decoded
3	Audio modelling	Audio CNN multi-label benchmarks	Complete	Pilot and expanded audio experiments completed
4	Hybrid modelling	Structured + audio hybrid benchmark	Complete	Expanded hybrid benchmark frozen as main candi...
5	Full-genre extension	Full-161 Stage-1 + Stage-2 pipeline design	Complete	Candidate labels predicted directly; rare-tail...
6	Deployment demo	Single-file inference demonstration	Complete	One-file end-to-end full-161 inference completed
7	Deployment demo	Batch inference demonstration	Complete	Random batch and targeted rare-tail batch demo...
8	Comparative reporting	Comparative performance summary tables	Complete	Project-level benchmark and fallback summaries...
9	Interpretation	Final limitations and next-step framing	Complete	Ready to translate into report and supervisor ...

In [9]:

```
# ============================================================
# 9. BUILD FINAL RECOMMENDATION TABLE
# ============================================================

recommendations_df = pd.DataFrame([
    {
        "Recommendation Type": "Main benchmark model",
        "Recommended System": "Expanded Hybrid Global Threshold",
        "Configuration": "Structured 0.1 | Audio 0.9 | Threshold 0.20",
        "Why": "Best candidate-150 benchmark performance"
    },
    {
        "Recommendation Type": "Conservative optional mode",
        "Recommended System": "Expanded Hybrid Per-Label + Best Cap",
        "Configuration": "Cap = 3",
        "Why": "Cleaner optional conservative decoding mode"
    },
    {
        "Recommendation Type": "Full-161 deployment configuration",
        "Recommended System": "Stage 1 benchmark + Stage 2 fallback",
        "Configuration": "Stage 1 benchmark plus baseline_prior @ 0.05 for Stage 
2",
        "Why": "Most practical final all-genre pipeline assembled in the project"
```

```
    },
    {
        "Recommendation Type": "How to describe Stage 2",
        "Recommended System": "Rare-tail fallback assistance layer",
        "Configuration": "Not a full standalone rare-tail classifier",
        "Why": "Rare-tail support remains sparse and should be presented honestly"
    }
])

print("Final recommendations:")
display(recommendations_df)
```

```
Final recommendations:
```

	Recommendation Type	Recommended System	Configuration	Why
0	Main benchmark model	Expanded Hybrid Global Threshold	Structured 0.1 | Audio 0.9 | Threshold 0.20	Best candidate-150 benchmark performance
1	Conservative optional mode	Expanded Hybrid Per-Label + Best Cap	Cap = 3	Cleaner optional conservative decoding mode
2	Full-161 deployment configuration	Stage 1 benchmark + Stage 2 fallback	Stage 1 benchmark plus baseline_prior @ 0.05 f...	Most practical final all-genre pipeline assemb...
3	How to describe Stage 2	Rare-tail fallback assistance layer	Not a full standalone rare-tail classifier	Rare-tail support remains sparse and should be...

In [10]:

```
# ============================================================
# 10. BUILD FINAL LIMITATIONS AND NEXT-STEPS TABLE
# ============================================================

limitations_next_steps_df = pd.DataFrame([
    {
        "Category": "Limitation",
        "Item": "Class imbalance and rare-tail sparsity",
        "Details": "Many rare-tail labels have very few examples, limiting direct
modelling reliability"
    },
    {
        "Category": "Limitation",
        "Item": "Stage-2 rare-tail behaviour remains weak relative to Stage-1",
        "Details": "Fallback routing is useful but should not be overstated as
strong rare-tail prediction"
    },
    {
        "Category": "Limitation",
        "Item": "Structured branch underperformed audio and hybrid branches",
        "Details": "Structured features were still useful for fusion, but not
```

```
strong enough alone"
    },
    {
        "Category": "Next Step",
        "Item": "Report writing and supervisor explanation",
        "Details": "Translate these tables into final report sections, visual
summaries, and discussion"
    },
    {
        "Category": "Next Step",
        "Item": "Presentation preparation",
        "Details": "Build final supervisor slides using frozen benchmark, pipeline,
and demo outputs"
    },
    {
        "Category": "Next Step",
        "Item": "Optional future research",
        "Details": "Explore better few-shot / metric-learning / hierarchical rare-
tail handling"
    },
    {
        "Category": "Next Step",
        "Item": "Optional future scaling",
        "Details": "Extend more aggressively beyond candidate-150 using stronger
rare-tail support strategies"
    }
])

print("Final limitations and next steps:")
display(limitations_next_steps_df)
```

```
Final limitations and next steps:
```

	Category	Item	Details
0	Limitation	Class imbalance and rare-tail sparsity	Many rare-tail labels have very few examples, ...
1	Limitation	Stage-2 rare-tail behaviour remains weak relat...	Fallback routing is useful but should not be o...
2	Limitation	Structured branch underperformed audio and hyb...	Structured features were still useful for fusi...
3	Next Step	Report writing and supervisor explanation	Translate these tables into final report secti...
4	Next Step	Presentation preparation	Build final supervisor slides using frozen ben...
5	Next Step	Optional future research	Explore better few-shot / metric-learning / hi...
6	Next Step	Optional future scaling	Extend more aggressively beyond candidate-150 ...

In [11]:

```
# ============================================================
# 11. BUILD FINAL EXECUTIVE SUMMARY TABLE
# ============================================================

executive_summary_df = pd.DataFrame([
    {
        "Section": "Dataset",
        "Summary": "FMA-Large used as primary working dataset with multi-label genre structure"
    },
    {
        "Section": "Storage and pipeline",
        "Summary": "Metadata, labels, and hierarchy prepared and stored through a reproducible pipeline with MongoDB support"
    },
    {
        "Section": "Best candidate-150 model",
        "Summary": "Expanded hybrid benchmark froze as the strongest direct prediction system"
    },
    {
        "Section": "Full-161 design",
        "Summary": "Stage 1 handles candidate labels directly; Stage 2 handles sparse rare-tail labels through fallback routing"
    },
    {
        "Section": "Deployment demos",
        "Summary": "Single-file, random-batch, targeted-rare-tail, and reranker demos completed"
    },
    {
        "Section": "Overall conclusion",
        "Summary": "The project successfully produced a scalable multi-label genre pipeline with a strong benchmark core and a limited but usable rare-tail fallback layer"
    }
])

print("Executive summary table:")
display(executive_summary_df)
```

```
Executive summary table:
```

	Section	Summary
0	Dataset	FMA-Large used as primary working dataset with...
1	Storage and pipeline	Metadata, labels, and hierarchy prepared and s...
2	Best candidate-150 model	Expanded hybrid benchmark froze as the stronge...
3	Full-161 design	Stage 1 handles candidate labels directly; Sta...
4	Deployment demos	Single-file, random-batch, targeted-rare-tail,...
5	Overall conclusion	The project successfully produced a scalable m...

In [12]:

```
# ============================================================
# 12. BUILD FINAL REPORT-READY PARAGRAPH BANK
# ============================================================

paragraph_bank_df = pd.DataFrame([
    {
        "Paragraph Type": "Results summary",
        "Text": (
            "The final candidate-150 benchmark system was the expanded hybrid 
configuration, "
            "which combined the structured and audio branches using weighted 
fusion. This model "
            "outperformed the individual structured and audio branches and was 
therefore frozen "
            "as the main benchmark system for the project."
        )
    },
    {
        "Paragraph Type": "Full-pipeline summary",
        "Text": (
            "To extend the project from the candidate-150 benchmark space toward 
the full 161-label "
            "genre inventory, a two-stage full-161 pipeline was assembled. Stage 1 
performed direct "
            "candidate-label prediction, while Stage 2 acted as a fallback layer 
for sparse rare-tail "
            "genres using hierarchy-aware routing."
        )
    },
    {
        "Paragraph Type": "Rare-tail interpretation",
        "Text": (
            "The rare-tail component should be interpreted as a fallback assistance 
layer rather than "
            "as a strong standalone classifier. The results showed that sparse 
labels remain difficult, "
            "but the project still produced a practical mechanism for surfacing 
limited rare-tail hints "
            "without overstating their reliability."
        )
    },
```

```
    {
        "Paragraph Type": "Project conclusion",
        "Text": (
            "Overall, the project achieved its main objective of building a reproducible, scalable genre "
            "classification workflow over the FMA dataset by combining metadata management, structured "
            "modelling, audio modelling, hybrid fusion, and deployment-style inference demonstrations."
        )
    }
])

print("Paragraph bank:")
display(paragraph_bank_df)
```

```
Paragraph bank:
```

	Paragraph Type	Text
0	Results summary	The final candidate-150 benchmark system was t...
1	Full-pipeline summary	To extend the project from the candidate-150 b...
2	Rare-tail interpretation	The rare-tail component should be interpreted ...
3	Project conclusion	Overall, the project achieved its main objecti...

In [13]:

```
# ============================================================
# 13. SAVE FINAL CONSOLIDATION OUTPUTS
# ============================================================

candidate150_final_df.to_csv(
    f"{PROCESSED_DIR}/final_project_candidate150_benchmark_table.csv",
    index=False
)

full161_final_df.to_csv(
    f"{PROCESSED_DIR}/final_project_full161_system_table.csv",
    index=False
)

performance_snapshot_df.to_csv(
    f"{PROCESSED_DIR}/final_project_performance_snapshot.csv",
    index=False
)

deliverables_df.to_csv(
    f"{PROCESSED_DIR}/final_project_deliverables_checklist.csv",
    index=False
)

recommendations_df.to_csv(
    f"{PROCESSED_DIR}/final_project_recommendations.csv",
```

```
    index=False
)

limitations_next_steps_df.to_csv(
    f"{PROCESSED_DIR}/final_project_limitations_next_steps.csv",
    index=False
)

executive_summary_df.to_csv(
    f"{PROCESSED_DIR}/final_project_executive_summary.csv",
    index=False
)

paragraph_bank_df.to_csv(
    f"{PROCESSED_DIR}/final_project_paragraph_bank.csv",
    index=False
)

if full161_pipeline_summary_df is not None:
    full161_pipeline_summary_df.to_csv(
        f"{PROCESSED_DIR}/final_project_loaded_full161_pipeline_summary_copy.csv",
        index=False
    )

if batch_demo_summary_df is not None:
    batch_demo_summary_df.to_csv(
        f"{PROCESSED_DIR}/final_project_loaded_batch_demo_summary_copy.csv",
        index=False
    )

if targeted_rare_tail_summary_df is not None:
    targeted_rare_tail_summary_df.to_csv(
        f"
{PROCESSED_DIR}/final_project_loaded_targeted_rare_tail_summary_copy.csv",
        index=False
    )

if stage2_reranker_test_comparison_df is not None:
    stage2_reranker_test_comparison_df.to_csv(
        f"
{PROCESSED_DIR}/final_project_loaded_stage2_reranker_test_comparison_copy.csv",
        index=False
    )

with open(f"{PROCESSED_DIR}/final_project_frozen_config.json", "w") as f:
    json.dump(final_project_config, f, indent=2)

print("Saved final full-project consolidation outputs.")
```

```
Saved final full-project consolidation outputs.
```

In [14]:

```
# ============================================================
# 14. INTERPRETATION NOTES
# ============================================================
```

```python
print("1. This notebook freezes the final state of the project.")
print("2. The expanded hybrid benchmark remains the main candidate-150 model.")
print("3. The full-161 system is framed as a two-stage pipeline with direct prediction plus rare-tail fallback.")
print("4. Stage-2 rare-tail behaviour should be described honestly as limited but usable fallback support.")
print("5. The saved tables can now be used directly for the final report, supervisor explanation, and presentation.")
```

```
1. This notebook freezes the final state of the project.
2. The expanded hybrid benchmark remains the main candidate-150 model.
3. The full-161 system is framed as a two-stage pipeline with direct prediction plus rare-tail fallback.
4. Stage-2 rare-tail behaviour should be described honestly as limited but usable fallback support.
5. The saved tables can now be used directly for the final report, supervisor explanation, and presentation.
```